10 Python Programs with Outputs

BTech Practical Programs | Simple Python code and sample runs

Note: The questions do not specify exact rates, limits, or policies. These programs use sample assumptions for demonstration. Change the values and conditions according to your faculty's requirements.

1. University Scholarship Allocation

Program

```
n = int(input("Number of students: "))
students = []

for i in range(n):
    name = input("Name: ")
    marks = float(input("Aggregate marks: "))
    subject = float(input("Minimum subject marks: "))
    attendance = float(input("Attendance: "))
    income = float(input("Family income: "))

    if marks >= 75 and subject >= 40 and attendance >= 75 and income <= 200000:
        students.append((name, marks))

students.sort(key=lambda x: (-x[1], x[0]))

print("\nEligible Students (Ranked):")
for rank, student in enumerate(students, 1):
    print(rank, student[0], "-", student[1])
```

Sample Output

```
Number of students: 2
Name: Anu
Aggregate marks: 85
Minimum subject marks: 60
Attendance: 90
Family income: 150000
Name: Ravi
Aggregate marks: 78
Minimum subject marks: 45
Attendance: 80
Family income: 180000

Eligible Students (Ranked):
1 Anu - 85.0
2 Ravi - 78.0
```

2. Progressive Electricity Billing

Program

```
units = int(input("Enter units: "))

if units <= 100:
    energy_charge = units * 2
elif units <= 200:
    energy_charge = 100 * 2 + (units - 100) * 3
else:
    energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

fixed_charge = 50
surcharge = energy_charge * 0.05
rebate = 20 if units < 100 else 0
bill = energy_charge + fixed_charge + surcharge - rebate

print("Energy charge:", energy_charge)
print("Fixed charge:", fixed_charge)
print("Surcharge:", round(surcharge, 2))
print("Rebate:", rebate)
print("Final bill:", round(bill, 2))
```

Sample Output

```
Enter units: 250
Energy charge: 650
Fixed charge: 50
Surcharge: 32.5
Rebate: 0
Final bill: 732.5
```

3. Hospital Billing and Patient Classification

Program

```
consultation = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))

total = consultation + diagnostic + treatment

if total >= 10000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount

if final_bill >= 10000:
    category = "High payment"
else:
    category = "Standard payment"

print("Total bill:", round(total, 2))
print("Discount:", round(discount, 2))
print("Final bill:", round(final_bill, 2))
print("Patient category:", category)
```

Sample Output

```
Consultation charge: 1000
Diagnostic charge: 3000
Treatment charge: 8000
Total bill: 12000.0
Discount: 1200.0
Final bill: 10800.0
Patient category: High payment
```

4. Employee Payroll and Deduction Analysis

Program

```
basic = float(input("Basic pay: "))
allowances = float(input("Allowances: "))
overtime = float(input("Overtime pay: "))
incentives = float(input("Incentives: "))

gross = basic + allowances + overtime + incentives

if gross <= 20000:
    deduction = gross * 0.05
else:
    deduction = 1000 + (gross - 20000) * 0.10

net_salary = gross - deduction

print("\n----- SALARY STATEMENT -----")
print("Basic pay:", basic)
print("Gross salary:", round(gross, 2))
print("Total deduction:", round(deduction, 2))
print("Net salary:", round(net_salary, 2))
```

Sample Output

```
Basic pay: 25000
Allowances: 5000
Overtime pay: 2000
Incentives: 1000

----- SALARY STATEMENT -----
Basic pay: 25000.0
Gross salary: 33000.0
Total deduction: 2300.0
Net salary: 30700.0
```

5. Multi-Plan Telecommunications Cost Optimization

Program

```
usage = float(input("Monthly data usage (GB): "))
budget = float(input("Budget: "))

# Each plan: name, base price, included GB, extra charge per GB
plans = [
    ("Plan A", 199, 10, 20),
    ("Plan B", 299, 20, 15),
    ("Plan C", 399, 30, 10)
]

eligible = []

for name, price, limit, extra_rate in plans:
    excess = max(0, usage - limit)
    total_cost = price + excess * extra_rate
    if total_cost <= budget:
        eligible.append((total_cost, name))

if eligible:
    eligible.sort()
    print("Most economical eligible plan:", eligible[0][1])
    print("Total cost:", round(eligible[0][0], 2))
else:
    print("No plan is within the budget.")
```

Sample Output

```
Monthly data usage (GB): 18
Budget: 350
Most economical eligible plan: Plan B
Total cost: 299
```

6. Railway Fare and Reservation Calculator

Program

```
distance = float(input("Distance in km: "))
age = int(input("Passenger age: "))
travel_class = input("Class (Sleeper/AC): ").strip().lower()

if travel_class == "sleeper":
    rate = 2
elif travel_class == "ac":
    rate = 5
else:
    print("Invalid class")
    raise SystemExit

fare = distance * rate

if age >= 60:
    fare *= 0.80       # 20% concession

reservation_fee = 30
total_fare = fare + reservation_fee

print("Basic fare:", round(fare, 2))
print("Reservation fee:", reservation_fee)
print("Total fare:", round(total_fare, 2))
```

Sample Output

```
Distance in km: 200
Passenger age: 65
Class (Sleeper/AC): Sleeper
Basic fare: 320.0
Reservation fee: 30
Total fare: 350.0
```

7. Retail Billing with Multiple Discount Rules

Program

```
n = int(input("Number of products: "))
subtotal = 0

for i in range(n):
    print("Product", i + 1)
    price = float(input("Price: "))
    quantity = int(input("Quantity: "))
    subtotal += price * quantity

if subtotal >= 5000:
    discount = min(subtotal * 0.10, 1000)
elif subtotal >= 2000:
    discount = subtotal * 0.05
else:
    discount = 0

taxable_amount = subtotal - discount
tax = taxable_amount * 0.05
final_bill = taxable_amount + tax

print("Subtotal:", round(subtotal, 2))
print("Discount:", round(discount, 2))
print("Tax:", round(tax, 2))
print("Final bill:", round(final_bill, 2))
```

Sample Output

```
Number of products: 2
Product 1
Price: 1500
Quantity: 2
Product 2
Price: 1000
Quantity: 1
Subtotal: 4000.0
Discount: 200.0
Tax: 190.0
Final bill: 3990.0
```

8. Manufacturing Cost and Profitability Analysis

Program

```
material = float(input("Material cost: "))
labour = float(input("Labour cost: "))
machine = float(input("Machine cost: "))
wastage = float(input("Wastage cost: "))
selling_price = float(input("Total selling price: "))
target_profit = float(input("Required profit target: "))

total_cost = material + labour + machine + wastage
profit = selling_price - total_cost

print("Total production cost:", round(total_cost, 2))
print("Profit:", round(profit, 2))

if profit >= target_profit:
    print("Profit target achieved")
else:
    print("Profit target not achieved")
```

Sample Output

```
Material cost: 5000
Labour cost: 2000
Machine cost: 1000
Wastage cost: 500
Total selling price: 11000
Required profit target: 2000
Total production cost: 8500.0
Profit: 2500.0
Profit target achieved
```

9. Multi-Currency Travel Budget Analysis

Program

```
budget_inr = float(input("Travel budget in INR: "))
exchange_rate = float(input("INR per 1 foreign currency unit: "))
conversion_fee_percent = float(input("Conversion fee percentage: "))

categories = ["Food", "Travel", "Hotel"]
total_inr = 0
largest_category = ""
largest_amount = -1

for category in categories:
    foreign_amount = float(input(category + " expense in foreign currency: "))
    amount_inr = foreign_amount * exchange_rate
    total_inr += amount_inr

    if amount_inr > largest_amount:
        largest_amount = amount_inr
        largest_category = category

fee = total_inr * conversion_fee_percent / 100
grand_total = total_inr + fee

print("Expenses in INR:", round(total_inr, 2))
print("Conversion fee:", round(fee, 2))
print("Total spending:", round(grand_total, 2))
print("Largest expense category:", largest_category)

if grand_total <= budget_inr:
    print("Within budget")
else:
    print("Budget exceeded")
```

Sample Output

```
Travel budget in INR: 50000
INR per 1 foreign currency unit: 90
Conversion fee percentage: 2
Food expense in foreign currency: 100
Travel expense in foreign currency: 200
Hotel expense in foreign currency: 250
Expenses in INR: 49500.0
Conversion fee: 990.0
Total spending: 50490.0
Largest expense category: Hotel
Budget exceeded
```

10. Campus Event Financial Planning

Program

```
venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
catering = float(input("Catering cost: "))
transport = float(input("Transport cost: "))
certificates = float(input("Certificate cost: "))
sponsorship = float(input("Sponsorship amount: "))
registration_fee = float(input("Registration fee per participant: "))

total_cost = venue + equipment + catering + transport + certificates
shortfall = max(0, total_cost - sponsorship)

if registration_fee > 0:
    import math
    minimum_participants = math.ceil(shortfall / registration_fee)
else:
    minimum_participants = 0 if shortfall == 0 else -1

print("Total event cost:", round(total_cost, 2))
print("Sponsorship shortfall:", round(shortfall, 2))

if minimum_participants >= 0:
    print("Minimum registrations needed:", minimum_participants)
else:
    print("Registration fee must be greater than zero.")
```

Sample Output

```
Venue cost: 10000
Equipment cost: 5000
Catering cost: 12000
Transport cost: 3000
Certificate cost: 1000
Sponsorship amount: 10000
Registration fee per participant: 200
Total event cost: 31000.0
Sponsorship shortfall: 21000.0
Minimum registrations needed: 105
```